

Deployment Strategies

Intermediate Kubernetes

Module 11 of 13





Agenda

Core Strategies

- Rolling updates (default)
- Rollback and revision history
- Blue-green deployments
- Canary deployments

Advanced Topics

- Progressive delivery tooling
- A/B testing & shadow launches
- Deployment safety mechanisms
- EKS best practices & CI/CD

Rolling Updates

The Default Deployment Strategy



How Rolling Updates Work

Incremental Pod Replacement

Kubernetes gradually replaces old ReplicaSet pods with new ones, controlled by two parameters: `maxSurge` and `maxUnavailable`.

`maxSurge`

- Extra pods above desired count
- Absolute number or percentage
- Default: **25%**
- Higher = faster rollout, more resources

`maxUnavailable`

- Pods that can be down during update
- Absolute number or percentage
- Default: **25%**
- Lower = safer but slower rollout



Rolling Update Configuration

```
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # At most 5 pods during update
      maxUnavailable: 0 # Zero downtime
    revisionHistoryLimit: 10
  minReadySeconds: 10 # Wait 10s before Ready
# ... template, container spec, readinessProbe ...
```



Monitoring Rollout Progress

```
# Watch rollout status in real time
kubectl rollout status deployment/api-server -n production

# View rollout history
kubectl rollout history deployment/api-server -n production

# Inspect a specific revision
kubectl rollout history deployment/api-server \
  --revision=3 -n production

# Pause a rollout mid-way (investigate issues)
kubectl rollout pause deployment/api-server -n production

# Resume after investigation
kubectl rollout resume deployment/api-server -n production
```

Tip: Use `--watch` with `kubectl get pods` to see individual pod transitions during a rollout. Combine with `-l app=api-server` to filter.

Pause is powerful: Pausing a rollout mid-way creates a partial canary — some pods run the old version, some the new. Use this for manual verification before resuming.

Rollback and Revision History

```
# Undo to previous revision
kubectl rollout undo deployment/api-server -n production

# Undo to a specific revision
kubectl rollout undo deployment/api-server \
  --to-revision=2 -n production

# Annotate deployments for better history
kubectl annotate deployment/api-server \
  kubernetes.io/change-cause="Upgrade to v2.1.0 - JIRA-4521" \
  -n production
```

Fast Recovery: Rollback reuses the old ReplicaSet — no image pull needed if cached. Recovery takes seconds, not minutes.

Database Caveat: Rollback reverts pods but not database migrations. Always ensure backward-compatible schema changes.



Blue-Green Deployments

Instant Cutover with Full Rollback

Blue-Green Architecture

Core Principle

Run two identical environments in parallel. Route all traffic to one (blue). Deploy changes to the other (green). Switch traffic atomically when ready.

✓ Advantages

- Instant rollback (switch back)
- No mixed versions serving
- Full pre-flight testing

⚠ Trade-offs

- 2x resource cost during deploy
- Database compatibility required
- Session state must be external

Best for: Critical services, compliance-heavy environments, APIs with

Blue-Green Setup

Blue Deployment

```
# Deployment: api-blue (current version)
spec:
  replicas: 4
  selector:
    matchLabels: { app: api-server, slot: blue }
  template:
    metadata:
      labels: { app: api-server, slot: blue, version: v2.
# ... container spec ...
```

Pattern: Both Deployments share app: api-server. The slot label differentiates them.

Service Switch

```
apiVersion: v1
kind: Service
metadata:
  name: api-server
spec:
  selector:
    app: api-server
    slot: blue # Switch to "green"
  ports:
    - port: 80
      targetPort: 8080
```

```
# Atomic cutover
kubectl patch svc api-server \
  -p '{"spec":{"selector":{"slot":"green"}}}' \
  -n production
```

Atomic switch: Changing the Service selector instantly redirects all new connections to green pods. Existing connections drain naturally.



Blue-Green with Ingress / Gateway API

```
# HTTPRoute (Gateway API)
spec:
  rules:
    - backendRefs:
      - name: api-blue
        port: 80
        weight: 100      # Flip weights for cutover
      - name: api-green
        port: 80
        weight: 0
```

Gateway API advantage: Weight-based routing enables gradual cutover (90/10 → 50/50 → 0/100) rather than all-or-nothing. Blurs the line between blue-green and canary.

Canary Deployments

Controlled Risk with Gradual Traffic Shifting

Canary Concept

Progressive Traffic Shifting

Deploy the new version alongside the current version. Route a small percentage of traffic to it. Observe metrics. Gradually increase traffic or roll back.

Canary Progression

- **Step 1:** 5% traffic to canary
- **Step 2:** Observe error rates, latency
- **Step 3:** Promote to 25%, then 50%
- **Step 4:** Full promotion to

Key Metrics to Watch

- HTTP error rate (5xx responses)
- Request latency (p50, p95, p99)
- Resource consumption (CPU, memory)



Manual Canary Deployments

Stable (4 replicas)

```
# Deployment: api-stable
spec:
  replicas: 4
  selector:
    matchLabels: { app: api-server, track: stable }
  template:
    metadata:
      labels: { app: api-server, track: stable }
    # ... container spec ...
```

The Service selects on app: api-server, matching both stable and canary pods.

Canary (1 replica)

```
# Deployment: api-canary
spec:
  replicas: 1 # ~20% traffic (1 of 5 total)
  selector:
    matchLabels: { app: api-server, track: canary }
  template:
    metadata:
      labels: { app: api-server, track: canary }
    # ... different image tag ...
```

Limitation: Traffic split is tied to pod ratio (1/5 = 20%). For finer control, use ingress-level splitting.



Canary with NGINX Ingress Annotations

```
# Canary Ingress – references same host as primary
metadata:
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "10"
    # Also: canary-by-header, canary-by-cookie
spec:
  rules:
    - host: api.example.com
      # ... paths route to api-canary Service
```



Progressive Delivery: Argo Rollouts

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: api-server
spec:
  strategy:
    canary:
      steps:
        - setWeight: 5
        - pause: { duration: 5m }
        - setWeight: 25
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
  canaryService: api-canary
  stableService: api-stable
```

Argo Rollouts replaces Deployments with a custom Rollout CRD. It provides built-in canary steps, analysis templates, and service mesh integration.



Progressive Delivery: Flagger

```
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: api-server
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-server
  service:
    port: 80
  analysis:
    interval: 1m
    threshold: 5
    maxWeight: 50
    stepWeight: 10
    metrics:
      - name: request-success-rate
        threshold: 99
```

Key difference: Flagger wraps *existing* Deployments and automates canary analysis based on metrics. No CRD replacement needed.

Advanced Strategies

A/B Testing, Shadow Launches & Feature Flags



A/B Testing with Header-Based Routing

```
# NGINX Ingress: header-based routing
metadata:
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-by-header: "X-Test-Group"
    nginx.ingress.kubernetes.io/canary-by-header-value: "experiment-b"
---
# Gateway API: header match
spec:
  rules:
    - matches:
        - headers:
            - { name: X-Test-Group, value: experiment-b }
      backendRefs:
        - { name: api-experiment-b, port: 80 }
    - backendRefs:
        - { name: api-control, port: 80 }
```



Shadow / Dark Launches

Traffic Mirroring

Duplicate live traffic to the new version. The shadow processes requests but its responses are discarded.

```
# Istio VirtualService - traffic mirroring
spec:
  http:
    - route:
        - destination: { host: api-stable }
      mirror:
        host: api-shadow
        mirrorPercentage:
          value: 100.0
```

Zero risk to users: Shadow receives real traffic patterns but responses are discarded. Validates performance and correctness before any user exposure.

Feature Flags vs Deployment Strategies

Deployment Strategies

- Control at **infrastructure** level
- Entire application version changes
- Operated by platform team
- Rollback = redeploy old version
- Scope: all-or-nothing per pod

Feature Flags

- Control at **application** level
- Toggle individual features in code
- Operated by development team
- Rollback = flip the flag off
- Scope: per-feature, per-user

Recommended pattern: Combine both. Use deployment strategies for infrastructure changes and feature flags for feature activation. Deploy dark (flag off), then enable progressively.



Deployment Safety

Guardrails for Safe Releases



Pod Disruption Budgets

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-server-pdb
  namespace: production
spec:
  minAvailable: 3           # Always keep at least 3 pods
  # OR: maxUnavailable: 1   # Allow at most 1 pod down
  selector:
    matchLabels:
      app: api-server
```

Voluntary disruptions: PDBs protect against node drains, cluster autoscaler, and maintenance operations.

Not for crashes: PDBs do not protect against involuntary disruptions like node failures or OOM kills.

During deployments: PDBs interact with rolling updates. A PDB of `minAvailable: 3` on a 4-replica Deployment effectively forces `maxUnavailable: 1`, regardless of the Deployment's own strategy configuration.



Deployment Conditions & Failure Detection

```
# Set a deadline for rollout completion
spec:
  progressDeadlineSeconds: 300 # 5-minute deadline
  minReadySeconds: 15         # Pod must be Ready for 15s
```

```
# Check deployment conditions
kubectl get deployment api-server -o json | \
  jq '.status.conditions[] | select(.type=="Progressing")'

# Example output for a stuck rollout:
# {
#   "type": "Progressing",
#   "status": "False",
#   "reason": "ProgressDeadlineExceeded",
#   "message": "has timed out progressing"
# }
```

Stuck rollouts: If a new pod fails readiness probes repeatedly, the rollout stalls. `progressDeadlineSeconds` marks it as failed after the deadline, enabling automated alerting and rollback.



CI/CD Pipeline Integration Patterns

Deployment Pipeline Flow

Build → Test → Image Push → Deploy to Staging → Integration Tests →
Deploy → Verify → Promote or Rollback

```
# GitOps deployment pattern (ArgoCD / Flux)
# 1. Update image tag in Git repo
git commit -m "Release api-server v2.1.0"
git push origin main

# 2. ArgoCD detects change and applies
argocd app sync api-server --prune

# 3. Wait for rollout to complete
kubectl rollout status deployment/api-server \
  -n production --timeout=300s

# 4. Run smoke tests
curl -sf https://api.example.com/healthz || \
  kubectl rollout undo deployment/api-server -n production
```

GitOps principle: Git is the single source of truth. All deployment changes go through Git, enabling audit trails, approvals, and automated rollback to any previous commit.

EKS Deployment Best Practices

Node Management

- Use managed node groups for automatic AMI updates
- Configure PDBs before node group updates
- Use Karpenter for rapid scaling during rollouts

Image Management

- Use ECR with image scanning enabled
- Immutable tags (never reuse `:latest`)
- Pre-pull images on nodes before rollout

Observability

- CloudWatch Container Insights for metrics
- AWS X-Ray for tracing during canary
- ALB access logs for traffic analysis

EKS tip: When using ALB Ingress Controller for canary deployments, configure `alb.ingress.kubernetes.io/actions.weighted-`



Strategy Comparison Matrix

Strategy	Downtime	Resource Cost	Rollback Speed	Complexity
Rolling Update	None	Low (+25%)	Medium (new rollout)	Low (built-in)
Blue-Green	None	High (2x)	Instant (switch back)	Low-Medium
Canary	None	Low-Medium	Fast (kill canary)	Medium-High
A/B Testing	None	Medium	Fast (routing change)	High
Shadow	None	High (2x)	N/A (no user impact)	High (mesh required)

Choosing a strategy: Start with rolling updates. Add canary for critical services. Use blue-green when instant rollback is non-negotiable. Layer in feature flags for granular control.



Module Summary

- Rolling updates: zero-downtime with tunable maxSurge/maxUnavailable
- Blue-green: instant rollback via Service selector switching
- Canary: reduced blast radius with gradual traffic shifting
- PDBs protect availability during voluntary disruptions
- progressDeadlineSeconds catches stuck rollouts



Lab 11: Deployment Strategies

Hands-On Exercise

- Configure a rolling update with custom maxSurge and maxUnavailable
- Monitor a rollout and perform a rollback to a specific revision
- Implement a blue-green deployment with Service label switching
- Implement canary deployment with replica ratios
- Set up Pod Disruption Budgets and test with a dry-run node drain
- Implement progressDeadlineSeconds and trigger a failed rollout

Duration: ~45 minutes

Key Takeaways

1. Rolling updates are the baseline. Tune maxSurge, maxUnavailable, and minReadySeconds. Add canary for critical services.

2. Blue-green provides instant rollback. Worth 2x resource cost for critical services. Canary reduces blast radius with metric-driven promotions.

3. Safety mechanisms are not optional. PDBs, progressDeadlineSeconds, readiness probes, and revisionHistoryLimit should be on every Deployment. Combine with feature flags to decouple deployment from release.

? Questions & Discussion

Module 11: Deployment Strategies

Up Next: Module 12 — Helm & Templating